Hochschule Zittau/Görlitz
UNIVERSITY OF APPLIED SCIENCES

Polling & Long Polling

Web Engineering 1 (319300)

Vorlesung 04

M.Sc. Christopher-Manuel Hilgner

Hochschule Zittau/Görlitz

christopher.hilgner@stud.hszg.de

2026-10-27

Agenda

- Polling & Long Polling
- Web Sockets

— Agenda

- Begrüßung, kurzer Rückblick auf Vorlesung 03 (Fetch-API, async/await)
- Lernziel: Die drei Grundansätze kennen und deren Vor- und Nachteile begründen können
- Zeitplan: ca. 30 Minuten Polling/Long Polling, Rest Web Sockets

Polling & Long Polling

Das Problem

- HTTP ist ein **Request-Response-Protokoll**: Der Client fragt, der Server antwortet
- Der Server kann von sich aus **nichts** an den Browser schicken
- Frage: Wie erfährt die Seite von neuen Daten (Chatnachricht, Benachrichtigung, Auftragsstatus)?

Drei Ansätze mit steigender Komplexität:

Ansatz	Idee
Polling	Client fragt in festen Abständen nach
Long Polling	Server hält die Anfrage offen, bis es etwas Neues gibt
Server-Sent Events, Web Sockets	Dauerhafte Verbindung (kommt später)

Einstiegsfrage an das Auditorium: Wer hat schon mal einen Chat im Browser benutzt – wie kommt die neue Nachricht eigentlich auf den Bildschirm? Die Tabelle bewusst nur kurz vorstellen, Details folgen auf den nächsten Folien.

Polling

- Der Client sendet in regelmäßigen Abständen eine Anfrage, egal ob es etwas Neues gibt
- Einfachste Variante
- Funktioniert mit jedem Server und jedem Proxy

Analogie: Der Client klingelt alle paar Sekunden an der Tür und fragt „Gibt's was Neues?“. Schon hier andeuten: Die meisten Anfragen kommen leer zurück – genau das ist der Nachteil, der auf der Bewertungsfolie auftaucht. Robustheit ist der eigentliche Verkaufspunkt: keine Spezialtechnik nötig, läuft auch in restriktiven Firmennetzwerken.

Polling

Beispiel

```JavaScript
async function pollStatus() {
  try {
    const response = await fetch("/api/orders/42/status");
    if (!response.ok) throw new Error("HTTP ${response.status}");
    const { status } = await response.json();
    updateUI(status);
  } catch (error) {
    console.error(error);
  }
}
setInterval(pollStatus, 5000); // alle 5 Sekunden
```


Code gemeinsam durchgehen: fetch, Fehlerprüfung über response.ok, Destrukturierung aus der JSON-Antwort, updateUI als Blackbox lassen. Frage ans Auditorium: Was passiert, wenn der Server länger als 5 Sekunden braucht? – leitet direkt zur nächsten Folie über.

Polling Umsetzung

Problem von `setInterval`: Dauert eine Antwort länger als das Intervall, überlappen sich beide Anfragen.

Besser: rekursives `setTimeout`. Die nächste Anfrage startet erst, wenn die letzte fertig ist.

Weitere Verbesserungen:

- Polling stoppen, wenn der Tab im Hintergrund ist (`document.visibilityState`, Event `visibilitychange`)
- Bei Fehlern das Intervall vergrößern (Exponential Backoff: 5s, 10s, 20s, …)
- Abbruch über `AbortController`, z.B. beim Verlassen der Seite
- Serverseitig `ETag`/`If-None-Match` nutzen: Antwort `304 Not Modified` spart Daten

Kernproblem kurz an der Tafel skizzieren: zwei zeitlich überlappende Anfragen bei setInterval. Die Verbesserungen der Liste nur streichen – jede davon wird im Praktikum relevant. ETag/304 an die HTTP-Grundlagen anbinden, die Studenten kennen Conditional Requests bereits.

Polling Umsetzung

```JavaScript
async function poll() {
  try {
    const response = await fetch("/api/orders/42/status");
    if (!response.ok) throw new Error("HTTP ${response.status}");
    const { status } = await response.json();
    updateUI(status);
  } catch (error) {
    console.error(error);
  } finally {
    setTimeout(poll, 5000);
  }
}
poll();
```


Gegenüberstellung mit dem ersten Beispiel: Der Unterschied steckt komplett im finally-Block – setTimeout startet erst, wenn die vorherige Anfrage abgeschlossen ist (auch im Fehlerfall). Serielle Ausführung statt möglicher Parallelität.

Bewertung

Vorteile:
- Sehr einfach zu implementieren
- Funktioniert überall (Proxys, Firewalls, Caches)
- Server bleibt zustandslos

Nachteile
- Latenz bis zu einem Intervall (bei 5s im Schnitt 2.5s Verzögerung)
- Viele unnötige Requests, wenn sich selten etwas ändert
- Kurzes Intervall = Extra Last auf Server und Netz
- Langes Intervall = Träge UI

Typischer Einsatz: Dashboards, Statusabfragen von langlaufenden Jobs, wenn Verzögerungen von Sekunden egal sind

Rechenfrage ans Auditorium: Warum im Schnitt 2,5 Sekunden Verzögerung bei einem 5-Sekunden-Intervall? (Event tritt gleichverteilt zwischen zwei Polls auf.) Trade-off kurzes/langes Intervall an den Nachteilen festmachen. Bekanntes Beispiel als Einsatzfall: Statusseite eines Deployments oder Paketverfolgung.

Long Polling

Long Polling

Idee: Der Server beantwortet die Anfrage nicht sofort, sondern hält sie offen, bis es etwas Neues gibt (oder ein Timeout erreicht ist). Danach öffnet der Client sofort die nächste Anfrage.

Ablauf

1. Client sendet Anfrage
2. Server wartet (z.B. bis zu 30s)
3. Neue Daten da: Server antwortet sofort. Timeout: Server antwortet leer (z.B. 204 No Content)
4. Client verarbeitet die Antwort und startet direkt die nächste Anfrage

Mentalmodell: Der Server „schläft“ im Request-Handler ein und wird von den neuen Daten geweckt. Die vier Schritte nummeriert durchgehen. Wichtig: Das Timeout (leere 204er-Antwort) sorgt dafür, dass Proxys die Verbindung nicht vorher abwürgen – dazu mehr auf den nächsten Folien.

Long Polling

```JavaScript
async function longPoll(lastId = 0) {
  while (true) {
    try {
      const response = await fetch("/path?after=${lastId}");
      if (response.status == 200) {
        const messages = await response.json();
        messages.forEach(show);
        lastId = messages.at(-1).id;
      }
      // 204 = Timeout ohne Neuigkeiten, erneut fragen
    } catch (error) {
      console.error(error);
      await new Promise((resolve) => setTimeout(resolve,
      5000)); // Pause bei Fehler
```

Long Polling

```
14        }
15      }
16    }
17    longPoll();
```

Wichtig: Der Client merkt sich, was er schon hat (`lastId`), damit keine Nachricht zwischen zwei Anfragen verloren geht.

Long Polling
— Long Polling

Fokus auf lastId: Das Cursor-Prinzip schließt die Lücke, die zwischen zwei Anfragen entsteht – eine Nachricht, die genau dazwischen am Server ankommt, wird mit dem nächsten Poll über „after“ noch geliefert. Kurz auf den Fehlerzweig eingehen: 5-Sekunden-Pause ist hier bereits einfachstes Backoff.

Long Polling bewertet

Vorteile:

- Fast sofortige Zustellung
- Weniger leere Requests als Polling
- Läuft über normales HTTP

Nachteile:

- Der Server hält viele offene Verbindungen
- Proxys und Load Balancer beenden lange Verbindungen (Timeout oft 30 bis 60s)
- Pro Domain gibt es Verbindungslimits (HTTP/1.1: meist 6 pro Browser)
- Zwischen zwei Anfragen entsteht kurz eine „Lücke“

Kontrast zu Polling herausarbeiten: Latenz fast null, aber der Preis sind dauerhaft offene Verbindungen auf dem Server. Das Verbindungslimit von 6 pro Domain ist praktisch relevant: mehrere Tabs und Services teilen sich dasselbe Budget. Die „Lücke“ wurde auf der Code-Folie schon mit dem lastId-Cursor gelöst.

Long Polling bewertet

- Server-Timeout kürzer als der Proxy-Timeout wählen
- Server muss viele wartende Anfragen effizient halten können (asynchrones/ nicht blockierendes I/O, bei Thread-pro-Request-Modellen wird das sehr schnell eng)
- Bei Fehlern immer mit Pause und Backoff neu verbinden, sonst entsteht eine „Thundering Herd“, wenn der Server wieder erreichbar ist

Faustregel zuerst: Server-Timeout unterhalb des Proxy-Timeouts wählen, sonst antwortet der Proxy mit einem Fehler, bevor der Server überhaupt dran war. „Thundering Herd“ erklären: Nach einem Ausfall feuern alle Clients gleichzeitig sofort wieder los – nur die Pause mit Backoff verteilt die Last.

Vergleich und Ausblick

	Polling	Long Poll.	SSE	WebSocket
Richtung	Client → Server	Client → Server	Server → Client	beidseitig
Latenz	Intervall	gering	gering	gering
Verbindung	neu pro Anfrage	neu pro Antwort	dauerhaft	dauerhaft
Protokoll	HTTP	HTTP	HTTP (`EventSource`)	eigenes (`ws://`, `wss://`)
Aufwand	sehr gering	mittel	gering	höher

Tabelle zeilenweise durchgehen. Die Spalten SSE und WebSocket sind bewusst noch Blackbox – Vorschau auf die kommende Vorlesung. An der Zeile „Richtung“ festigen: Bei Polling und Long Polling initiiert immer der Client die Kommunikation, bei SSE dreht sich das um.

Vergleich und Ausblick

Faustregel:

- Selten ändernde Daten, Sekunden egal: **Polling**
- Neuigkeiten sollen schnell ankommen, nur Server → Client: **SSE** (oder Long Polling als Fallback)
- Echte Zwei-Wege-Kommunikation (Chat, Spiele, Kollaboration): **WebSocket**

Kurzer Ausblick auf SSE:

```javascript
const source = new EventSource("/api/events");
source.onmessage = (event) =>
console.log(JSON.parse(event.data));
```


Kurzquiz zu den Faustregeln: Welche Technik für (a) Paketverfolgung, (b) Live-Ticker, (c) Multiplayer-Spiel? Das EventSource-Snippet nur als Ausblick zeigen, kein Code-Walkthrough – Details folgen in der nächsten Vorlesung.

Typische Fehler

- `setInterval` mit `async`-Funktion: Anfragen überlappen sich, wenn der Server langsam ist
- **Kein Abbruch**: Polling läuft weiter, obwohl die Seite oder Komponente längst weg ist (Memory Leak, vgl. Folie zu Garbage Collection)
- **Zu kurze Intervalle**: 1-Sekunden-Polling von 1000 Nutzern sind 1000 Requests pro Sekunde
- **Endlosschleife ohne Pause bei Fehlern**: Der Client hämmert dann den Server mit Fehlversuchen zu

Typische Review-Funde – gut als Abschlussdiskussion geeignet: Welchen der vier Fehler habt ihr schon selbst gebaut? Beim Stichwort Memory Leak an die Vorlesung zu Garbage Collection anknüpfen. Bei „1-Sekunden-Polling von 1000 Nutzern“ die Hochrechnung kurz vorführen (1000 Requests/s).

Web Sockets

Web Sockets